LEARNER LAB DOCUMENT · DAY 14

Incident Response — Detection

Lab: Incident Response — Detection with Wazuh **Role:** Junior SOC Analyst **Estimated time:** 90 minutes **Curriculum position:** Day 14 of the cybersecurity bootcamp

1. Lab Name

Incident Response — Detection with Wazuh

2. Lab Purpose

This lab teaches you how a Security Operations Center (SOC) analyst detects suspicious activity using a centralized Security Information and Event Management (SIEM) platform. You will use Wazuh — an open-source SIEM and Extended Detection and Response (XDR) platform — to observe how attacker actions on a monitored Windows server produce alerts in near real-time, and how those alerts map to the MITRE ATT&CK framework.

You will play both sides of the engagement: from a Kali Linux workstation you will simulate two attacker behaviors — a Remote Desktop Protocol (RDP) password-guessing attack with hydra, and an attempt to mount a Windows administrative share with both invalid and valid credentials. After each action, you will pivot to the Wazuh dashboard to confirm that the alert fired, capture the Rule ID, and interpret the associated tactic and technique. Finally, you will observe how an attacker's attempt to clear the Windows Security event log is itself an event that Wazuh detects.

The point is not to "hack" the target. The point is to understand, from the SOC analyst's seat, what attacker activity looks like in a SIEM, which rules catch it, and how to read those rules to drive an incident response.

In scope: authenticated and unauthenticated activity against the lab-provided DC10 server only, simulated brute-force using the provided wordlist, share-mount attempts with the provided credentials, log inspection inside Wazuh. **Out of scope:** any activity against hosts other than DC10, modifying Wazuh rules, persistence or post-exploitation actions on DC10 beyond the steps listed.

3. Business Scenario

You are a Junior SOC Analyst on your team's day shift. Your team lead has assigned you a guided exercise on the lab tenant before letting you handle production alerts. The goal is to make you comfortable with three things: (1) what common attacker activity looks like when it hits the SIEM, (2) how to read Wazuh's Rule IDs and MITRE ATT&CK mappings, and (3) why centralized logging matters when an attacker tries to cover their tracks.

Your team lead wants the following:

1. Confirm your Kali workstation is ready and that you can authenticate to the Wazuh dashboard.
2. Prepare a password list and run an RDP password-guessing attack against DC10 using hydra.
3. Identify the Wazuh alerts that fired for the brute-force activity and document their Rule IDs, tactics, and techniques.
4. Attempt to mount a Windows administrative share with both invalid and valid credentials, and correlate each attempt to its Wazuh alert.
5. From DC10 itself, clear the Windows Security event log and confirm that Wazuh detects the deletion.
6. Summarize, in your report, what each detection tells the SOC about attacker behavior and where Wazuh's default MITRE mapping might need analyst verification.

Your deliverable is a short triage report plus screenshots showing the Wazuh alerts for each phase.

4. Skills Developed by the Lab

- Authenticating to a Wazuh dashboard and navigating the Security events view
- Filtering Wazuh alerts by agent (monitored host) and Rule ID
- Preparing a custom password list for an authentication-attack tool
- Running hydra against an RDP service on a Windows target
- Interpreting hydra output to distinguish failed attempts from successful guesses
- Reading Wazuh alert details: Level, Rule ID, Description, Tactic, Technique
- Cross-referencing Wazuh's MITRE ATT&CK mappings against the actual observed behavior
- Mounting a Windows administrative share from Linux using cifs-utils
- Correlating host-side activity to SIEM alerts across multiple Rule IDs
- Recognizing log-clearing as an anti-forensics technique (MITRE T1070)
- Validating that centralized logging captures activity even when the local host log is cleared
- Documenting incident detection findings in a structured triage report

5. Learning Objectives

By the end of this lab, you will be able to:

1. Authenticate to a Wazuh dashboard and filter Security events to a single monitored agent.
2. Prepare a wordlist and execute an authenticated password-guessing attack with hydra against an RDP service.
3. Identify the Wazuh Rule IDs that correspond to failed and successful logon attempts, and capture their MITRE ATT&CK tactic and technique mappings.
4. Mount a Windows administrative share from Linux using both invalid and valid credentials and observe the resulting alerts.
5. Correlate host activity to SIEM alerts by Rule ID and timestamp.
6. Identify, from inside Wazuh, that a Windows Security event log has been cleared on a monitored host.
7. Explain why centralized logging is essential when an attacker attempts log clearing as an anti-forensics measure.
8. Document the full chain of detection in a triage report suitable for hand-off to a Tier 2 analyst.

6. Lab Architecture and Requirements

6.1 Lab Topology

This lab runs entirely on your local Windows machine. The Kali attacker VM and the DC10 target VM are **nested virtual machines** hosted in VMware Workstation, both running on top of Windows 11.

```
+-----------------------------------------------------------+
|  Your Physical Machine — Windows 11 (Host Operating System)|
|                                                           |
|   +---------------------------------------------------+   |
|   |          VMware Workstation / Player              |   |
|   |                                                   |   |
|   |   +------------------+    +------------------+    |   |
|   |   |   Kali Linux VM  |    |    DC10 (Win)    |    |   |
|   |   |   attacker side  |    |   target server  |    |   |
|   |   |   + Wazuh agent  |<->|   + Wazuh agent  |    |   |
|   |   +------------------+    +------------------+    |   |
|   |             |                        ^            |   |
|   |             |  Internal Lab Network |            |   |
|   |             |    (VMnet, isolated)  |            |   |
|   |             v                        |            |   |
|   |   +----------------------------------------+    |   |
|   |   |       Wazuh Manager / Dashboard VM     |    |   |
|   |   |       https://wazuh.lab                |    |   |
|   |   |       admin / Pa??w0rd                 |    |   |
|   |   +----------------------------------------+    |   |
|   |                                                   |   |
|   +---------------------------------------------------+   |
|                                                           |
|   Firefox on Windows 11 reaches the Wazuh dashboard       |
|   either directly (NAT-bridged) or by RDP into Kali first |
+-----------------------------------------------------------+
```

6.2 Target details

Item	Value
Primary target	DC10 (Windows Server, Domain Controller)
Target IP (lab-allocated)	`10.10.10.10` *(verify with* `ping dc10` *before use)*
Target services in scope	RDP (3389/tcp), SMB administrative share (`C$`)
Wazuh dashboard URL	`https://wazuh.lab` *(or the IP shown in your VMware network)*
Wazuh credentials	`admin` / `Pa??w0rd`
Valid Windows admin credentials	`Administrator` / `Pa$$w0rd`
Invalid Windows credentials (for failure case)	`Jaime` / `Pa$$w0rd`
Host OS	Windows 11 (your physical machine)
Hypervisor	VMware Workstation / Player
Authorization context	Local nested-VM lab, isolated network, instructor-authorized

> **Note for instructors:** the Wazuh password contains two question-mark characters in positions 3 and 4 (`Pa??w0rd`). The Windows password uses two dollar signs in the same positions (`Pa$$w0rd`). These differ deliberately. Re-verify both before each cohort.

6.3 Lab requirements

- **Host OS (your physical machine):** Windows 11 with VMware Workstation or VMware Player installed, virtualization enabled in BIOS/UEFI (Intel VT-x / AMD-V), at least 16 GB of RAM available to the host.
- **Nested VMs (provided as `.ova` or `.vmx` bundles):** Kali Linux attacker VM, DC10 Windows Server target VM, Wazuh Manager VM.
- **Tools (pre-installed on the Kali VM):** Firefox, terminal, hydra, cifs-utils (`mount.cifs`), nano or vi.
- **Tools on DC10:** Event Viewer (`eventvwr.msc`).
- **Working folder (on Kali):** `~/ir_detection_lab/`
- **Pre-staged files in the working folder:**

- `report_template.md`
- `screenshots/` (empty directory)
- A copy of `/usr/share/wordlists/seclists/Passwords/Common-Credentials/500-worst-passwords.txt` is available; you will copy and modify it into `passlist.txt`.

6.4 Accessing the lab environment

1. On your Windows 11 host, launch **VMware Workstation** (or VMware Player).
2. Open the lab VM library and **power on** the three VMs in this order: **Wazuh Manager**, then **DC10**, then **Kali Linux**. Wait roughly 60 seconds between Wazuh and DC10 so the agent has time to register.
3. Click into the Kali VM console window and sign in as `kali` / `kali`. Maximize the VM window and confirm a terminal is reachable.
4. From the Kali desktop, confirm you can reach the other VMs:
   - `ping -c 2 10.10.10.10` (DC10)
   - Open Firefox to the Wazuh dashboard URL and accept the self-signed certificate warning.
5. You will sign into the DC10 VM directly (via the VMware console for DC10) later in the lab as `Administrator` / `Pa$$w0rd`. Do not RDP into DC10 from Kali for the manual steps — use the VMware console window.

> **Watch the passwords carefully.** The Wazuh password is `Pa??w0rd` (two question marks). The Windows password is `Pa$$w0rd` (two dollar signs). They are different on purpose.

6.5 Pre-flight verification

Before starting Task 1, confirm:

- All three VMs are powered on and have finished booting.
- The working folder `~/ir_detection_lab/` exists on Kali and is writable.
- The `screenshots/` subfolder exists and is empty.
- The `report_template.md` file is present.
- `ping -c 2 10.10.10.10` (or `ping dc10`) returns replies — Kali can reach DC10.
- `hydra -h` runs without error.
- `which mount.cifs` returns a path.
- Firefox launches and can load the Wazuh dashboard URL (you may need to accept a self-signed certificate warning).

If any of these fail, fix the issue before continuing.

---

7. Tasks

This is an **unassisted, hands-on lab**. Each task below tells you the **purpose**, any **constraints or scope rules**, what to **deliver**, how to **verify** you are done, what to **record**, and what **evidence** to submit. The task does not give you the commands — that is the assessment.

If you get stuck, the *Hints Companion* document offers hints, sample outputs, and common-mistake warnings for each task. Using it is optional but expected for entry-level learners.

Complete the tasks in order — each builds on the previous one.

Task 1 — Verify the Kali workstation and reach the target

Purpose: Confirm your attacker workstation is functional and can reach DC10 over the network before you generate any traffic that needs to appear in Wazuh.

Required coverage: Capture the Kali hostname, the Kali IP address on the lab interface, and a successful ping (or equivalent reachability test) to DC10.

Output file: `~/ir_detection_lab/task01_kali_identity.txt`

Verification step: The file contains the strings `inet`, the DC10 IP, and `bytes from` (or equivalent reply line). `wc -l` returns at least 10 lines.

Information to record:

- Kali hostname
- Kali IP address and interface name
- DC10 IP address (as resolved or as configured)
- Round-trip time observed in the reachability test

Evidence to submit: `task01_kali_identity.txt`, `screenshots/task01_terminal.png`

Task 2 — Prepare the password list

Purpose: Brute-force tools need a wordlist. You will copy a known dictionary file into your working folder and place the lab password at a specific line position so that hydra finds it during the attack.

Required coverage / Constraints:

- Start from a copy of `500-worst-passwords.txt` (do not edit the original).
- Save the working list as `~/ir_detection_lab/passlist.txt`.
- Ensure `Pa$$w0rd` appears in the file at a known line number (record which line).
- The file must contain at least 100 candidate passwords in total.

> **Scope reminders:** Do not edit the original wordlist under `/usr/share/wordlists/`. Work on your local copy only.

Output file: `~/ir_detection_lab/passlist.txt`

Verification step: `wc -l passlist.txt` shows at least 100 lines, and `grep -n 'Pa$$w0rd' passlist.txt` returns exactly one match with the line number you recorded.

Information to record:

- Source dictionary path
- Line number where `Pa$$w0rd` was inserted
- Total line count of `passlist.txt`

Evidence to submit: `passlist.txt`, `screenshots/task02_wordlist_grep.png`

Task 3 — Authenticate to the Wazuh dashboard and filter to DC10

Purpose: A SOC analyst's first move on shift is to scope the view to the asset they are investigating. You will log into Wazuh, navigate to Security events, and filter the view so that only events from DC10 are shown.

Required coverage:

- Successful authentication as `admin`.
- The Security events page (also called Threat Hunting in some Wazuh builds) is open.
- The agent filter is set so that only events from the DC10 agent are visible.

> **Scope reminders:** Do not modify any Wazuh rules, decoders, or configuration. You are a read-only investigator for this lab.

Output file: none (screenshot only)

Verification step: The screenshot clearly shows the Wazuh **Security events** page header, an active filter chip referencing the DC10 agent, and at least one event row in the table.

Information to record:

- Wazuh dashboard URL used
- DC10 agent ID as displayed in Wazuh
- Approximate event count visible for DC10 in the last 24 hours

Evidence to submit: `screenshots/task03_wazuh_dc10_filter.png`

Task 4 — Run a password-guessing attack against RDP on DC10

Purpose: Generate the attacker traffic that the SOC needs to detect. You will use hydra to attempt RDP logons against DC10 using a fixed username and your password list.

Required coverage / Constraints:

- Target service: RDP on DC10.
- Username: `Administrator`.
- Password source: your `passlist.txt` from Task 2.
- The attack must produce at least one successful guess (because `Pa$$w0rd` is in the list).
- Capture hydra's output to a file.

> **Scope reminders:** Run hydra against DC10 only. Do not point it at any other host on the lab network. Do not run hydra against external internet hosts or against your Windows 11 host machine.

Output file: `~/ir_detection_lab/task04_hydra_output.txt`

Verification step: The output file contains the string `password:` on the line that announces the successful guess, and the recovered password matches `Pa$$w0rd`.

Information to record:

- Exact command issued (for the report)
- Number of attempts hydra made before success
- Time taken from start to first valid result
- The recovered username/password pair

Evidence to submit: `task04_hydra_output.txt`, `screenshots/task04_hydra_success.png`

Task 5 — Identify the Wazuh alerts for the brute-force activity

Purpose: Pivot from the attacker view to the defender view. You will refresh the Wazuh Security events page for DC10 and identify the alerts that fired in response to your hydra run.

Required coverage:

- Locate at least one alert with **Rule ID 60122** (failed Windows logon) generated during your hydra run.
- Locate the alert with **Rule ID 92652** (successful login after multiple failed attempts — possible password guessing) for the same time window.
- Drill into the details panel for each rule and capture: Level, Description, Tactic(s), Technique(s).

Output file: none (screenshots and report entries only)

Verification step: Two screenshots exist — one showing a 60122 alert's detail view, one showing a 92652 alert's detail view. Each shows the Rule ID, Level, Description, and the MITRE tactic/technique fields populated.

Information to record:

- Count of 60122 alerts seen in the relevant time window
- Rule Level (numeric) for 60122 and for 92652
- The MITRE Tactic and Technique IDs Wazuh assigns to each rule
- The full Description text for 92652

Evidence to submit: `screenshots/task05_rule_60122.png`, `screenshots/task05_rule_92652.png`

Task 6 — Examine the MITRE ATT&CK mapping for Rule 92652

Purpose: Wazuh links its rules to the MITRE ATT&CK framework, but the default mapping is not always a perfect description of the observed attack. A SOC analyst must verify, not just accept.

Required coverage:

- From the 92652 alert detail, click through to the linked Technique page.
- Read the technique's name, ID (e.g. `T1110` family), and short description.
- Note whether the default mapping accurately reflects what hydra did against RDP. If you think the mapping is incomplete or slightly off, say so in your report and explain why.

Output file: none (report entry and screenshot only)

Verification step: The screenshot shows the MITRE technique page (either inside Wazuh's intelligence panel or on `attack.mitre.org`), and your report contains a one-paragraph analyst's note on the mapping's accuracy.

Information to record:

- Technique ID and name as linked from Wazuh
- Tactic the technique falls under
- Your one-paragraph analyst note on mapping accuracy

Evidence to submit: `screenshots/task06_mitre_technique.png`

Task 7 — Attempt to mount the DC10 administrative share with invalid credentials

Purpose: Different attacker behaviors produce different SIEM alerts. You will attempt to mount the DC10 `C$` administrative share from Kali using a username that does not exist, and observe the failure both at the command line and in Wazuh.

Required coverage / Constraints:

- Tool: `mount -t cifs` (or `mount.cifs`).
- Credentials: username `Jaime`, password `Pa$$w0rd`.
- Share: `\\DC10\C$` (or `//10.10.10.10/C$`).
- Mount point: a directory you create under `/mnt/` for this lab.
- Capture the full stderr/stdout to a file.

> **Scope reminders:** Mount only the DC10 `C$` share. Do not attempt to mount shares on any other host, including your Windows 11 host machine.

Output file: `~/ir_detection_lab/task07_mount_invalid.txt`

Verification step: The output file contains a permission-denied or logon-failure message, and the share is **not** mounted (verify with `mount | grep cifs`).

Information to record:

- Exact command issued
- Exact error message returned
- Local time of the attempt (for correlation with Wazuh)

Evidence to submit: `task07_mount_invalid.txt`

Task 8 — Mount the DC10 administrative share with valid credentials

Purpose: Repeat the same action with valid credentials so you can compare the failure and success paths side-by-side in Wazuh.

Required coverage / Constraints:

- Same tool, same share, same mount point as Task 7.
- Credentials: username `Administrator`, password `Pa$$w0rd`.
- Once mounted, list the root of the share to confirm access, then unmount cleanly.

Output file: `~/ir_detection_lab/task08_mount_valid.txt`

Verification step: The output file shows a successful directory listing of the share (visible Windows folders such as `Users`, `Windows`, `Program Files`), and a final `mount | grep cifs` showing the share is no longer mounted after cleanup.

Information to record:

- Exact command issued
- Local time of the successful mount
- Three folder names you observed in the share root

Evidence to submit: `task08_mount_valid.txt`, `screenshots/task08_share_listing.png`

Task 9 — Correlate the mount attempts to Wazuh alerts

Purpose: Map your Task 7 and Task 8 actions to their SIEM detections, and capture the Rule IDs Wazuh used to distinguish a failed logon from a successful one.

Required coverage:

- Locate an alert with **Rule ID 60122** generated by the failed `Jaime` mount.
- Locate an alert with **Rule ID 60106** (successful Windows logon) generated by the successful `Administrator` mount.
- Capture the MITRE tactic and technique each rule is mapped to (you should see `T1078` — Valid Accounts — appear).

Output file: none (screenshots and report entries only)

Verification step: Two screenshots, one per Rule ID, each showing the alert detail panel with Rule ID, Description, Level, Tactic, and Technique fields visible.

Information to record:

- Rule Level (numeric) for 60106 versus 60122
- The MITRE technique reference (e.g. `T1078`) for each
- Approximate Wazuh-recorded timestamp of each event

Evidence to submit: `screenshots/task09_rule_60122_mount.png`, `screenshots/task09_rule_60106_mount.png`

Task 10 — Clear the Windows Security event log on DC10

Purpose: Attackers commonly try to destroy local evidence after they gain access. You will perform that anti-forensics action on DC10 itself, then confirm in the next task that Wazuh's centralized logging captured the deletion even though the local log is gone.

Required coverage / Constraints:

- Open the DC10 VM console directly in VMware on your Windows 11 host.
- Sign in to DC10 as `Administrator` / `Pa$$w0rd`.
- Open Event Viewer (`eventvwr.msc`).
- Locate the **Windows Logs > Security** log.
- Clear the log (without saving a copy is acceptable for this exercise).
- Confirm the Security log is now empty (or contains only the new "log cleared" record).

> **Scope reminders:** Clear only the Security log. Do not clear the System or Application logs. Do not run wevtutil with a wildcard. Do not disable any Wazuh agent service.

Output file: none (screenshot only — from inside DC10)

Verification step: The Event Viewer screenshot shows the Security log either empty or containing only a fresh "The audit log was cleared" entry (Event ID 1102).

Information to record:

- Local time on DC10 when you cleared the log
- The Windows account used (which should be `Administrator`)

Evidence to submit: `screenshots/task10_dc10_log_cleared.png`

Task 11 — Confirm Wazuh detected the log-clearing event

Purpose: Demonstrate the central thesis of centralized logging: even when the local Windows Security log is destroyed, the SIEM still has the record because the event was forwarded by the Wazuh agent before deletion.

Required coverage:

- In Wazuh, locate a recent alert with **Rule ID 63103** (or the closest equivalent in your Wazuh ruleset describing "audit log was cleared" / Windows Event 1102).
- Capture the alert's Description, Level, Tactic, and Technique.
- The technique should map to `T1070` (Indicator Removal) or a sub-technique.

Output file: none (screenshot only)

Verification step: Screenshot shows the alert detail with the Rule ID, the source agent (DC10), the description referencing audit-log clearing, and a MITRE technique field referencing `T1070`.

Information to record:

- Exact Rule ID seen (record the actual ID you find, even if it differs slightly from 63103)
- Rule Level
- Wazuh-recorded timestamp of the event
- The MITRE technique displayed

Evidence to submit: `screenshots/task11_wazuh_log_cleared.png`

Task 12 — Write the incident detection report

Purpose: Convert your evidence into a triage-ready document a Tier 2 analyst could pick up and act on.

Required coverage:

- Open `report_template.md` and fill in every section.
- Build a chronological timeline of all attacker actions and their corresponding Wazuh alerts (Rule IDs and timestamps).
- Identify at least two distinct MITRE techniques observed.
- Include an analyst comment on at least one MITRE mapping you believe is incomplete or imperfect (from Task 6).
- Include a short "Why centralized logging matters" paragraph based on Task 11.
- Save the completed report as `ir_detection_report.md` (or `.docx`).

Output file: `~/ir_detection_lab/ir_detection_report.md`

Verification step: Every numbered subsection in the template has been replaced with real content; no placeholder `_em-dash text_` remains. The timeline contains at least 5 entries.

Information to record: (in the report itself)

Evidence to submit: `ir_detection_report.md`

---

8. Deliverables

Submit the following files inside a single archive named `ir_detection_lab_<yourname>.zip`:

1. `task01_kali_identity.txt`
2. `passlist.txt`
3. `task04_hydra_output.txt`
4. `task07_mount_invalid.txt`
5. `task08_mount_valid.txt`
6. `ir_detection_report.md`
7. `screenshots/task01_terminal.png`

8. `screenshots/task02_wordlist_grep.png`
9. `screenshots/task03_wazuh_dc10_filter.png`
0. `screenshots/task04_hydra_success.png`
1. `screenshots/task05_rule_60122.png`
2. `screenshots/task05_rule_92652.png`
3. `screenshots/task06_mitre_technique.png`
4. `screenshots/task08_share_listing.png`
5. `screenshots/task09_rule_60122_mount.png`
6. `screenshots/task09_rule_60106_mount.png`
7. `screenshots/task10_dc10_log_cleared.png`
8. `screenshots/task11_wazuh_log_cleared.png`

---

9. Submission Instructions

1. From the Kali terminal, change into the parent of your lab folder and create a zip: `zip -r ir_detection_lab_<yourname>.zip ir_detection_lab/`.
2. From the Kali VM, copy the zip to a shared folder or USB pass-through to your Windows 11 host (or use VMware shared folders).
3. From your Windows 11 host, open the bootcamp portal in a browser and sign in with your cohort credentials.
4. Open the **Day 14 — Incident Response: Detection** lab page and use the **Upload Deliverable** button to select your zip file.
5. Confirm the upload completes and the submission status shows **Submitted — Pending Review**.

If the portal is unavailable at submission time, email the zip to your team lead and post the filename and SHA-256 hash in the cohort channel.

---

10. Learner Report Template

Copy the section below into `ir_detection_report.md` and fill it in.

```
# Incident Detection Report — Lab 14

## 1. Analyst information
- Name: _Your name_
- Date: _Today's date_
- Cohort: _Your cohort_

## 2. Target information
- Target host: _DC10_
- Target IP: _value_
- Wazuh agent ID for DC10: _value_

## 3. Workstation identity
- Kali hostname: _value_
- Kali IP: _value_
- Reachability to DC10 confirmed at: _timestamp_

## 4. Password list
- Source dictionary: _path_
- Total lines: _count_
- Line number of `Pa$$w0rd`: _value_

## 5. Hydra brute-force attack (Task 4)
- Command issued: _command_
- Attempts before success: _count_
- Time to first valid result: _duration_
- Recovered credentials: _user / password_

## 6. Wazuh alerts — brute-force phase (Task 5)
- Rule 60122 — count, level, description: _value_
- Rule 92652 — level, description, MITRE tactic, MITRE technique: _values_

## 7. MITRE mapping analyst note (Task 6)
_One paragraph: does Wazuh's default technique mapping for Rule 92652 accurately describe what hydra did over RDP? Wh

## 8. Share-mount attempts (Tasks 7 and 8)
- Invalid attempt — command, error, time: _values_
- Valid attempt — command, observed folders, time: _values_

## 9. Wazuh alerts — share-mount phase (Task 9)
- Rule 60122 (failure) — level, timestamp, technique: _values_
- Rule 60106 (success) — level, timestamp, technique: _values_

## 10. Log clearing and Wazuh detection (Tasks 10 and 11)
- Local time of log clear on DC10: _value_
- Wazuh Rule ID detected: _value_
- Rule level and MITRE technique: _values_

## 11. Detection timeline
| Time | Source | Action | Wazuh Rule ID |
|---|---|---|---|
| _t1_ | Kali | hydra RDP attack start | — |
| _t2_ | Wazuh | first 60122 alert | 60122 |
| _t3_ | Wazuh | 92652 success alert | 92652 |
| _t4_ | Kali | failed share mount (Jaime) | — |
| _t5_ | Wazuh | 60122 alert (mount failure) | 60122 |
| _t6_ | Kali | successful share mount (Administrator) | — |
| _t7_ | Wazuh | 60106 alert (mount success) | 60106 |
| _t8_ | DC10 | Security log cleared | — |
| _t9_ | Wazuh | log-clear alert | _ID_ |

## 12. Key risks identified
- _Risk 1_
- _Risk 2_
- _Risk 3_

## 13. Recommended next steps
- _Recommendation 1_
- _Recommendation 2_
- _Recommendation 3_

## 14. Why centralized logging matters
_Short paragraph drawn from Task 11: explain what would have been lost if Wazuh were not forwarding events off-host._

## 15. Evidence list
_List every file in the submitted zip._
```